MODULE 7

Coding with Claude Code: Refactor & Debug

Duration	Format	Audience	Day / Position
45 min	**Lecture + Demo**	**Mixed Engineers**	**Day 1 / Seventh**

LEARNING OBJECTIVES

By the end of this module, participants will be able to:

1. Apply in-terminal and VS Code prompting best practices for refactoring and debugging tasks with Claude Code
2. Refactor existing Python code for testability using Claude Code while maintaining minimal diffs and preserving behavior
3. Auto-generate docstrings, type hints, and inline documentation for existing codebases using Claude Code
4. Debug Python errors by pasting tracebacks into Claude Code and following its diagnostic and fix workflow
5. Use checkpoints and /rewind to safely experiment with refactoring approaches without risking working code
6. Apply the iterative refactor-test-commit cycle to incrementally improve code quality with AI assistance

SESSION TIMING BREAKDOWN

0:00 – 0:08	**Section 1: Prompting Best Practices for Refactoring**
0:08 – 0:18	**Section 2: Refactoring for Testability with Minimal Diffs**
0:18 – 0:26	**Section 3: Auto-Generating Documentation**
0:26 – 0:37	**Section 4: Debugging from Tracebacks**
0:37 – 0:43	**Section 5: Safe Experimentation with Checkpoints**
0:43 – 0:45	**Wrap-Up: Quick Reference & Transition to Module 8**

0:00 – 0:08	SECTION 1: Prompting Best Practices for Refactoring

1. Prompting Best Practices for Refactoring

1.1 The Refactoring Mindset

Refactoring with Claude Code is fundamentally different from writing new code (Module 6). When you refactor, the code already works — your job is to make it better without breaking it. This requires more precise prompts, explicit constraints, and a safety-first workflow.

New Code Prompts	Refactoring Prompts
"Create a function that validates sensor data"	"Refactor validate_reading to reduce cyclomatic complexity without changing its return type or behavior"
Open-ended: describe what you want	Constrained: specify what must NOT change
Claude has creative freedom	Claude must preserve existing behavior
One prompt, one module	Incremental: one refactoring step at a time
Risk: wrong implementation	Risk: broken existing tests

1.2 The Golden Rules of Refactoring Prompts

Follow these rules every time you ask Claude Code to refactor:

✅ REFACTORING PROMPT RULES

- Always specify WHAT to change and WHAT to preserve: "Refactor X but keep the public API identical"
- Request minimal diffs: "Make the smallest change possible to achieve this"
- One refactoring per prompt: Don’t combine rename + extract + restructure in one ask
- Reference tests: "Ensure all existing tests in test_validators.py still pass after this change"
- Use @-references: "Refactor @src/sensor_toolkit/validators.py" so Claude reads the actual file
- Ask for explanation: "Explain what you changed and why" to verify Claude’s reasoning

1.3 Prompt Templates for Common Refactoring Tasks

```
# Extract a function
> Extract the temperature validation logic from
> validate_reading() in @src/sensor_toolkit/validators.py
> into a separate _validate_temperature() helper.
> Keep the public API unchanged. Run tests after.

# Rename with safety
> Rename the variable 'data' to 'sensor_readings'
> everywhere in @src/sensor_toolkit/cleaners.py.
> Update all references in tests/ as well.

# Reduce complexity
> The validate_reading function has 5 if-statements.
> Refactor it to use a list of (field, min, max) tuples
> and a loop. Keep the same validation behavior.
```

```
# Modernize patterns
> Replace all os.path usage in @src/sensor_toolkit/
> with pathlib.Path. Keep behavior identical.
```

0:08 – 0:18	SECTION 2: Refactoring for Testability with Minimal Diffs

2. Refactoring for Testability with Minimal Diffs

2.1 Why Testability Matters

Code that's hard to test is code that's hard to trust. Common testability problems include: functions that do too many things (high coupling), hidden dependencies (importing globals), and functions that read files or call APIs directly (side effects). Claude Code excels at restructuring code to fix these problems.

Testability Problem	Before (Hard to Test)	After (Easy to Test)
Function does too much	process_data() reads file + validates + transforms + saves	Split into: read_file(), validate(), transform(), save()
Hidden dependency	Uses global CONFIG dict inside function	Accept config as parameter with default
Side effects	Reads CSV file directly in function body	Accept file path or data as parameter
Hard-coded values	Thresholds embedded in if-statements	Extract to constants or function parameters
No error info	Returns True/False for validation	Returns list of error messages with details

2.2 Live Refactoring: Extract Dependencies

Let's refactor a common pattern from Lab 1 — a function that reads a CSV file and validates its contents:

```
# Before: Hard to test (reads file directly)
def validate_csv_file(filepath: str) -> dict:
    """Validate a CSV file of sensor readings."""
    import csv
    with open(filepath) as f:
        reader = csv.DictReader(f)
        errors = []
        for row in reader:
            if float(row['temperature']) > 150:
                errors.append(f"Row {reader.line_num}: temp too high")
    return {'valid': len(errors) == 0, 'errors': errors}
```

```
# Prompt to Claude Code:
> Refactor validate_csv_file in @src/sensor_toolkit/validators.py
> to separate file I/O from validation logic:
> 1. Extract a validate_rows(rows: list[dict]) function
>    that takes parsed row data (no file I/O)
> 2. Keep validate_csv_file as a thin wrapper that reads
>    the file and calls validate_rows
> 3. Update tests to test validate_rows directly
> 4. Keep the original validate_csv_file API unchanged
```

```
# After: Easy to test (pure function + thin wrapper)
```

```
def validate_rows(rows: list[dict]) -> dict:
    """Validate parsed sensor reading rows."""
    errors = []
    for i, row in enumerate(rows):
        if float(row['temperature']) > 150:
            errors.append(f"Row {i+1}: temp too high")
    return {'valid': len(errors) == 0, 'errors': errors}

def validate_csv_file(filepath: str) -> dict:
    """Read and validate a CSV file of sensor readings."""
    import csv
    with open(filepath) as f:
        rows = list(csv.DictReader(f))
    return validate_rows(rows)
```

☆ **KEY POINT:** The key insight: after refactoring, you can test validate_rows() with pure Python dicts — no file fixtures, no temp directories, no cleanup. Tests run faster, are more reliable, and cover more edge cases. The thin wrapper (validate_csv_file) needs only one integration test.

2.3 The Refactor-Test-Commit Cycle

Always follow this cycle when refactoring with Claude Code:

Step	What to Do
1. Baseline	Run tests first: !pytest tests/ -v — all must pass before refactoring
2. Checkpoint	Commit current state: git commit -am "pre-refactor baseline"
3. Refactor	Ask Claude to make ONE refactoring change
4. Test	Run tests immediately: !pytest tests/ -v
5. Review	Check the diff: !git diff — verify changes are minimal
6. Fix or Rewind	If tests fail: ask Claude to fix. If fix fails: /rewind
7. Commit	If tests pass: git commit -am "refactor: extract validate_rows"

0:18 – 0:26	SECTION 3: Auto-Generating Documentation

3. Auto-Generating Docstrings, Type Hints & Documentation

3.1 Why Auto-Documentation Matters

Legacy code and rapid prototypes often lack documentation. Manually adding docstrings and type hints to an existing codebase is tedious and error-prone. Claude Code can analyze function signatures, return values, and usage patterns to generate accurate documentation in seconds.

Documentation Type	Claude Code Prompt	When to Use
Google-style docstrings	> Add Google-style docstrings to all public functions in @src/sensor_toolkit/validators.py	After code is stable, before code review
Type hints	> Add type hints to all functions in @src/sensor_toolkit/cleaners.py including return types	During refactoring, for IDE support
Inline comments	> Add inline comments explaining the business logic in validate_reading(), not the code mechanics	For complex algorithms or domain logic
Module docstrings	> Add a module-level docstring to @src/sensor_toolkit/analyzers.py describing its purpose and public API	For package documentation
README updates	> Update README.md with usage examples for the sensor_toolkit package based on the current API	Before sharing with team or publishing

3.2 Batch Documentation Generation

```
# Add docstrings to an entire package at once:
> Add Google-style docstrings to all public functions
> and classes in src/sensor_toolkit/. Skip functions that
> already have docstrings. Include:
> - One-line summary
> - Args section with types and descriptions
> - Returns section with type and description
> - Raises section if the function raises exceptions

# Add type hints to files that lack them:
> Add type hints to all functions in
> @src/sensor_toolkit/cleaners.py. Use modern Python 3.10+
> syntax (list[str] not List[str], X | None not Optional[X]).
> Include return type annotations.

# Verify with ruff after adding type hints:
> !ruff check src/ --select ANN
```

☆ **KEY POINT:** Ask Claude to generate documentation AFTER the code is stable, not during initial development. If you add docstrings too early, every code change requires updating the docstrings too — double the work. The sweet spot is right before code review or merging to main.

3.3 Quality Checks for Generated Documentation

```
# Ask Claude to audit its own documentation:
> Review all docstrings in @src/sensor_toolkit/validators.py.
> Check that: parameter descriptions match actual behavior,
> return types match actual returns, and any raised exceptions
> are documented. Fix any inaccuracies.

# Check type hint consistency with mypy:
> !mypy src/sensor_toolkit/ --strict

# Have Claude explain a function to verify accuracy:
> Explain what validate_batch() does based on its
> implementation, not its docstring. Then compare with
> the docstring and fix any discrepancies.
```

⚠ **WARNING:** Claude generates documentation based on code analysis, but it can make mistakes — especially with complex business logic it hasn't seen before. Always review generated docstrings for accuracy. A wrong docstring is worse than no docstring because it actively misleads future developers.

0:26 – 0:37	SECTION 4: Debugging from Tracebacks

4. Debugging from Tracebacks: Paste Error, Get Fix

4.1 The Debug Workflow

Claude Code's debugging workflow is remarkably simple: paste the error, get a diagnosis and fix. But the quality of your debug prompt dramatically affects the quality of Claude's response.

Debug Prompt Quality	Example
Bad: vague	> My code doesn't work. Fix it.
Better: error only	> I'm getting a KeyError: 'sensor_id'. Fix it.
Good: error + context	> Running pytest tests/test_validators.py gives this error: [paste full traceback]. The test worked yesterday.
Best: error + context + constraint	> This traceback occurs when validate_batch receives an empty list. Fix it without changing the function signature. Run the test after fixing.

4.2 Paste-and-Fix Workflow

The most common debugging workflow with Claude Code:

```
# Step 1: Run the failing test
> !pytest tests/test_validators.py::test_empty_batch -v

# Step 2: Copy the traceback and paste it to Claude:
> This test is failing with the following error:
>
> FAILED tests/test_validators.py::test_empty_batch
> E   ZeroDivisionError: division by zero
>
> src/sensor_toolkit/validators.py:45: ZeroDivisionError
>
> The test calls validate_batch([]) with an empty list.
> Fix the bug without changing the public API.

# Step 3: Claude reads the file, finds line 45,
# identifies the division by zero (e.g., avg = sum/len
# where len is 0), and proposes a fix.

# Step 4: Run the test again:
> !pytest tests/test_validators.py::test_empty_batch -v
```

☆ **KEY POINT:** Always include: (1) the full traceback, not just the error message, (2) the command that triggered it, (3) what you expected to happen, and (4) any constraints on the fix. Claude uses the traceback to locate the exact file and line number, which makes its diagnosis far more accurate.

4.3 Advanced Debugging: Adding Loggers

For bugs that don't produce clear tracebacks (wrong output, intermittent failures), use Claude's logging approach:

```
# Ask Claude to add diagnostic logging:
> Add detailed debug logging to validate_batch() in
> @src/sensor_toolkit/validators.py. Log each reading's
> validation result and the final counts. Use Python's
> logging module at DEBUG level.

# Run with logging enabled:
> !python -c "import logging; logging.basicConfig(level=logging.DEBUG); from
sensor_toolkit.validators import validate_batch; validate_batch(test_data)"

# Paste the log output back to Claude:
> Here's the debug output. The counts show 5 valid but
> the function returns 4. Find the off-by-one error.

# After fixing, remove the debug logging:
> Remove all debug logging from validate_batch().
> Keep only the production log statements.
```

4.4 Debugging with VS Code Diagnostics

If you're using the VS Code extension, Claude automatically receives IDE diagnostics — linting errors, type mismatches, and import issues. This means Claude can proactively fix issues:

```
# In VS Code, Claude sees diagnostics automatically:
> Fix all the linting warnings and type errors that
> VS Code is showing in @src/sensor_toolkit/cleaners.py

# Claude sees the diagnostics panel and fixes:
# - Missing import for datetime
# - Type mismatch on line 23 (str vs int)
# - Unused variable 'temp' on line 45
# - ruff violation E501 (line too long) on line 67
```

Track A (IT Engineers)	Track B (Process Engineers)
Debug API errors: paste HTTP 500 responses with stack traces, debug async code with concurrent failures	Debug data pipeline errors: paste CSV parsing errors, debug wrong calculation results by adding logging
Debug CI failures: paste GitHub Actions log output and ask Claude to diagnose build/test failures	Debug file I/O errors: paste FileNotFoundError or PermissionError tracebacks with path context
Debug complex: multi-file bugs, race conditions, dependency conflicts	Debug simple: single-function bugs, type errors, off-by-one, null/None handling

0:37 – 0:43	SECTION 5: Safe Experimentation with Checkpoints

5. Safe Experimentation with Checkpoints

5.1 The Experiment Workflow

Refactoring is inherently risky — you're changing working code. Checkpoints (Module 4) and git provide a multi-layered safety net. Here's the workflow for safe experimentation:

Safety Layer	When to Use
Git commit	Before starting ANY refactoring session. This is your permanent safety net.
Checkpoints (automatic)	Claude creates these automatically before every change. Use /rewind to restore.
/rewind code only	When Claude's implementation is wrong but its reasoning was sound. Keep the conversation.
/rewind both	When Claude went in a completely wrong direction. Start fresh.
Fork from checkpoint	When you want to try an alternative approach without losing the current one.
git stash	When you want to temporarily set aside changes and try something else.

5.2 Practical: Try Two Approaches

A common scenario: you want to refactor a function but aren't sure which approach is better. Use checkpoints to try both:

```
# Step 1: Commit current state
> !git commit -am "checkpoint: before refactoring validators"

# Step 2: Try Approach A (extract helper functions)
> Refactor validate_reading to use extracted helper functions
> for each validation rule (_validate_temp, _validate_pressure, etc.)

# Step 3: Run tests and evaluate
> !pytest tests/test_validators.py -v
> Show me the diff: !git diff

# Step 4: Not happy? Rewind and try Approach B
> /rewind  (select "rewind code only")

# Step 5: Try Approach B (data-driven validation)
> Refactor validate_reading to use a VALIDATION_RULES dict
> mapping field names to (min, max) tuples. Loop over rules.

# Step 6: Compare and keep the winner
> !pytest tests/test_validators.py -v
> !git diff
```

> ☆ **KEY POINT:** This try-rewind-try workflow is the #1 productivity technique for refactoring. Instead of mentally debating which approach is better, you try both in minutes. Checkpoints make the cost of experimentation nearly zero.

5.3 When to Commit vs. When to Rewind

Situation	Action	Why
Tests pass, diff looks clean	git commit	Lock in the improvement permanently
Tests pass but diff is too large	Discuss with Claude, possibly rewind	Large diffs are risky and hard to review
Some tests fail after refactor	Ask Claude to fix the failures	Usually a minor issue in the refactoring
Many tests fail, code looks wrong	/rewind code only	Bad implementation but good intent
Claude went in completely wrong direction	/rewind both	Need to rethink the approach entirely
Want to try alternative approach	Fork from checkpoint	Explore without losing current work

0:43 – 0:45	WRAP-UP: Quick Reference & Transition to Module 8

6. Quick Reference & Transition

MODULE 7 RECAP

✅ Refactoring prompts must be constrained: specify what to change AND what to preserve. One refactoring per prompt.

✅ Refactor for testability: extract pure functions from I/O, accept dependencies as parameters, split large functions.

✅ Auto-generate documentation AFTER code is stable: Google-style docstrings, type hints (3.10+ syntax), module docstrings.

✅ Debug by pasting full tracebacks with context: include the command, expected behavior, and fix constraints.

✅ Advanced debugging: ask Claude to add loggers, run the code, paste log output back for iterative diagnosis.

✅ Safe experimentation: commit first, try approaches using /rewind, compare diffs, keep the winner.

QUICK REFERENCE CARD

Task	Prompt Pattern	Safety Step
Extract function	> Extract [logic] from [func] into [new_func]. Keep public API unchanged.	Run tests after
Add type hints	> Add type hints to all functions in @[file]. Use Python 3.10+ syntax.	Run mypy after
Add docstrings	> Add Google-style docstrings to public functions in @[file]. Skip existing ones.	Review accuracy
Debug traceback	> This test fails with [full traceback]. Fix without changing the function signature.	Run test after
Debug with logging	> Add debug logging to [func]. Run it. Paste output back.	Remove logging after fix
Rename safely	> Rename [old] to [new] in @[file] and update all references in tests/.	Run all tests
Reduce complexity	> Refactor [func] to reduce cyclomatic complexity. Keep behavior identical.	git diff to verify
Try two approaches	> [Approach A prompt]. Test. /rewind. [Approach B prompt]. Compare.	Commit baseline first
Fix linting errors	> Fix all ruff warnings in @[file]. Show me what changed.	Run ruff after
Modernize patterns	> Replace [old pattern] with [new pattern] in @[file]. Keep behavior identical.	Run full test suite

TRANSITION TO MODULE 8

You now know how to refactor and debug with Claude Code. Module 8 (AI-Driven Quality Assurance) takes testing to the next level — AI-driven test design with coverage prompts and edge case generation,

fixtures and mocks, flaky test triage, and property-based testing concepts. This builds directly on the test-fix-rewind cycle you practiced here.